# Proyecto de Arquitectura: Trazabilidad de Aguacate usando Blockchain

Kevin Andrés Castillo Dorado y Javier Alexis Dejesus Narvaez

13 de septiembre de 2026

## Instrucciones Aplicadas

Este documento consolida las 10 Historias de Usuario exigidas para el proyecto, distribuidas equitativamente (5 por integrante). Cada historia sigue la nomenclatura estándar, validaciones, RNFs (ISO 25010) y los patrones de diseño aplicados a la solución.

## Historias de Usuario: Kevin Andrés Castillo Dorado

### HU-01: Registrar lote de aguacate en blockchain

**Prioridad:** Alta
**2. Nomenclatura**
**Como:** Productor de la finca.
**Quiero:** Registrar un nuevo lote con su peso, finca y fecha de cosecha usando un contrato inteligente.
**Para:** Que los datos queden guardados en la blockchain y no se puedan alterar después.
**3. Validaciones (Criterios de Aceptación)**

1. El peso del lote tiene que ser mayor a 0 kg.
2. Hay que llenar todos los datos (finca, variedad, fecha y peso) antes de poder guardar.
3. El sistema debe revisar que la cuenta (billetera) de quien registra tenga permisos de Productor.
4. Si falla la red blockchain, debe salir un error y no guardar nada a medias.
5. Si todo sale bien (200 OK), el lote pasa a estado Registradoz se guarda el código de la transacción.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| Categoría ISO 25010 | Descripción del RNF | ¿Por qué es necesario? | ¿Dónde/Cómo se revisará? |
|---|---|---|---|
| Seguridad | Cifrar la conexión y firmar la petición al enviar los datos. | Para que no se roben la identidad del productor y sea seguro. | En el backend al revisar cómo se envía la transacción a la red. |
| Eficiencia | Que el sistema no tarde más de 4 segundos en responder. | Para que el usuario no se desespere esperando. | Con la consola del navegador al hacer clic en Registrar". |
| Tolerancia a Fallos | Poner un límite de tiempo (timeout) de 5 segundos. | Para que la página no se quede cargando infinito si el internet falla. | Apagando el internet justo al enviar los datos para ver qué pasa. |

**5. Patrones de Diseño Aplicados**

**Patrón Adapter (Estructural):** Lo usamos para conectar nuestro backend con la blockchain sin mezclar el código.

**Patrón DTO (Estructural):** Para pasar solo los datos necesarios desde la vista al controlador.

**6. Limitaciones del Alcance**

Esta historia NO incluye conectar sensores automáticos para pesar el aguacate.

**7. Consideraciones y Notas**

Hay que tener configurada la URL de la red de prueba (Testnet) en las variables de entorno.

**8. Diseño UI**

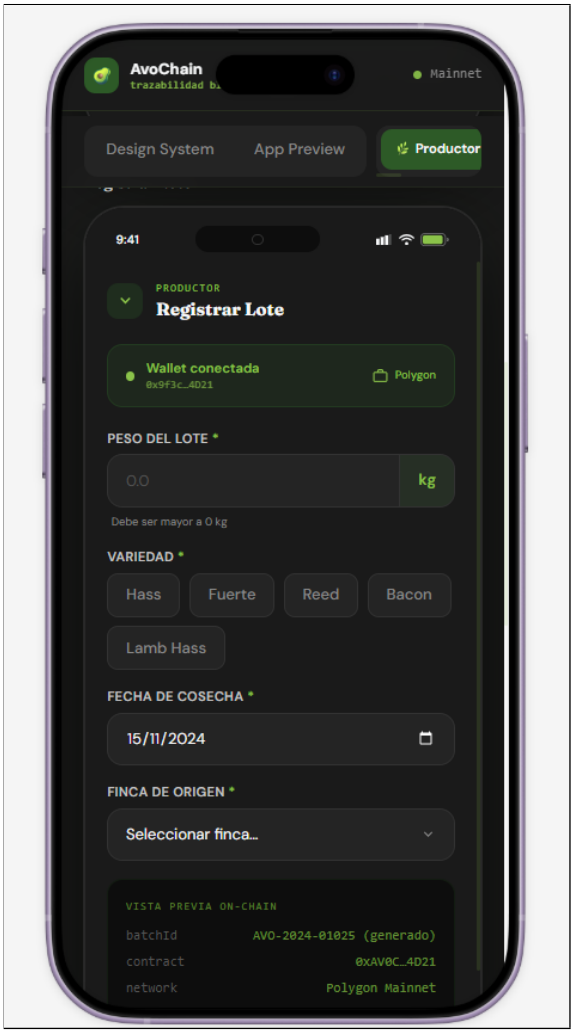



Figura 1: Vista móvil para registrar un nuevo lote en la blockchain.

### HU-02: Guardar fotos y detalles del producto

**Prioridad:** Media

**2. Nomenclatura**

**Como:** Productor de la finca.
**Quiero:** Subir fotos del aguacate recién cosechado.
**Para:** Tener evidencia visual de cómo sale el producto de la finca.

**3. Validaciones (Criterios de Aceptación)**

1. El sistema debe dejar subir mínimo 1 foto y máximo 5.
2. Las fotos no pueden pesar más de 5MB y deben ser JPG o PNG.
3. Si se cae el internet subiendo la foto, que deje intentar de nuevo sin borrar lo que ya se escribió.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| Categoría ISO 25010 | Descripción del RNF | ¿Por qué es necesario? | ¿Dónde/Cómo se revisará? |
|---|---|---|---|
| Escalabilidad | Guardar las imágenes en la nube (ej. AWS S3) y no en la base de datos. | Para no saturar nuestro servidor con fotos pesadas. | Revisando la URL donde queda guardada la imagen final. |
| Eficiencia | Que la foto suba rápido, en menos de 3 segundos. | Porque en las fincas el internet a veces es lento. | Midiendo el tiempo de carga en la pestaña Network. |

**5. Patrones de Diseño Aplicados**

**Patrón Factory Method:** Para manejar la subida de archivos sin importar si usamos Amazon o Google Drive de fondo.

**6. Limitaciones del Alcance**

No vamos a usar Inteligencia Artificial para verificar si la foto sí es de un aguacate.

**7. Consideraciones y Notas**

En la blockchain solo se guarda un código corto (hash) de la foto, la foto real se queda en el servidor para no gastar tanto.

**8. Diseño UI**

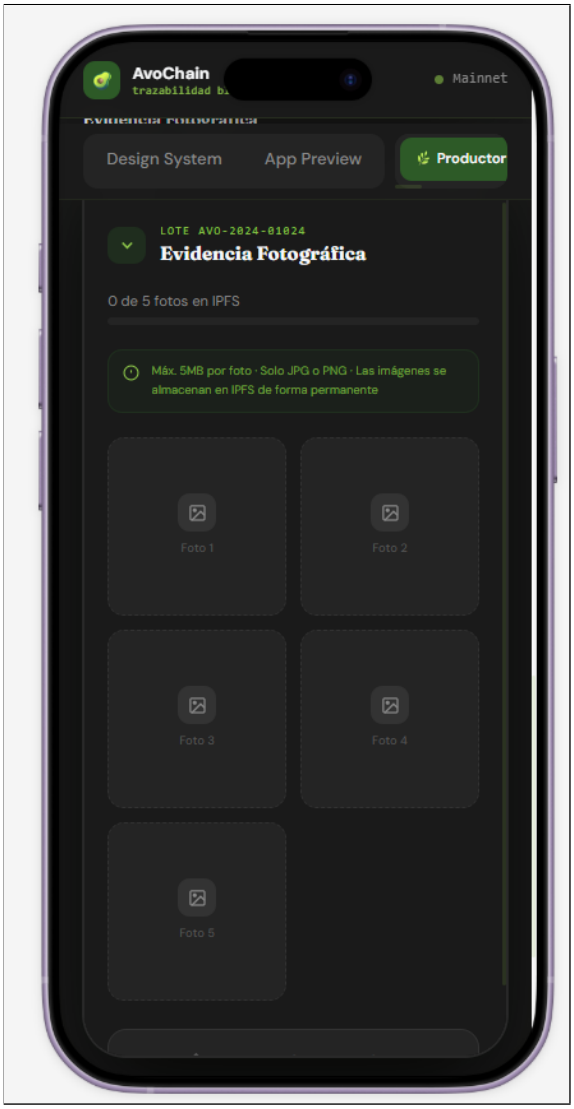



Figura 2: Vista móvil para la carga de evidencia fotográfica del producto.

### HU-03: Registrar el paso del producto en distribución

**Prioridad:** Alta

**2. Nomenclatura**

**Como:** Transportador.
**Quiero:** Escanear el código del aguacate al recibir la caja.
**Para:** Que quede constancia de que el producto ya va en camino y lo tengo yo.

**3. Validaciones (Criterios de Aceptación)**

1. Al escanear el código QR, el sistema debe confirmar que el lote sí existe.
2. Solo se puede escanear si el lote está en estado Registrado".
3. Debe guardar el usuario del conductor que escaneó la caja.

4. Mostrar una alerta si escanean una caja que no iba para esa ruta.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Disponibilidad | Que la app deje escanear sin internet y lo suba luego. | En la carretera casi nunca hay buena señal. | Quitándole los datos al celular y probando el escáner. |
| Mantenibilidad | Separar el código del lector de cámara del código del backend. | Por si después queremos cambiar la librería de escanear QR. | Revisando los diagramas de clases. |

**5. Patrones de Diseño Aplicados**

**Patrón State (Comportamiento):** Para manejar fácil el cambio del aguacate de .En Origen.a .En Tránsito".

**Patrón Proxy (Estructural):** Para interceptar el guardado y dejarlo en el celular si no hay internet.

**6. Limitaciones del Alcance**

NO incluye rastrear el camión por GPS en tiempo real.

**7. Consideraciones y Notas**

La aplicación móvil le va a pedir permiso de cámara al usuario la primera vez.

**8. Diseño UI**




Figura 3: Interfaz del conductor para el escaneo QR del lote en tránsito.

### HU-04: Consultar origen del producto en la tienda

**Prioridad:** Alta

**2. Nomenclatura**

**Como:** Cliente final en el supermercado.
**Quiero:** Escanear el código del aguacate con mi celular.
**Para:** Saber de qué finca viene y comprobar que no es falso.

**3. Validaciones (Criterios de Aceptación)**

1. Mostrar en pantalla la finca, fecha de cosecha y los transportistas.
2. Si escanean un código inventado, sacar una alerta grande de ”Producto Falso”.
3. Revisar que el estado del aguacate no aparezca como ”Vendido”.

4. Mostrar las fotos que subió el productor al inicio.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Eficiencia | Que la información cargue súper rápido (menos de 1.5 seg). | Para que la gente en el supermercado no se aburra esperando. | Revisando el tiempo de respuesta de la API. |
| Usabilidad | Interfaz clara y con letras grandes. | Para que se lea bien en cualquier celular. | Usando herramientas como Lighthouse de Chrome. |

**5. Patrones de Diseño Aplicados**

**Patrón Facade (Estructural):** Para juntar los datos que vienen de la base de datos normal y los que vienen de la blockchain, y mandarlos todos en una sola respuesta fácil al frontend.

**6. Limitaciones del Alcance**

Esta pantalla es solo informativa, no es para cobrarle el aguacate al cliente.

**7. Consideraciones y Notas**

Hacer consultas en la red blockchain es gratis, así que esto no gasta saldo de la billetera.

**8. Diseño UI**




Figura 4: Pantalla del cliente para consultar el origen y trazabilidad del producto.

### HU-05: Invalidar el código después de la venta

**Prioridad:** Alta

**2. Nomenclatura**

**Como:** Vendedor de la tienda.

**Quiero:** Marcar el aguacate como "Vendido.en el sistema.

**Para:** Quemar el código y evitar que alguien use la etiqueta vieja en un aguacate falso.

**3. Validaciones (Criterios de Aceptación)**

1. El sistema solo deja hacer esto si el aguacate está .En Punto de Venta".
2. Al venderlo, el estado en blockchain cambia a "Vendido".

3. Si alguien vuelve a escanear un aguacate vendido, bloquear y mostrar alerta roja de Çódigo Reutilizado".

4. Solo el vendedor con permisos puede hacer este paso.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Seguridad | Prevenir que un mismo código se pueda vender dos veces. | Para cuidar la confianza en el sistema. | Haciendo pruebas intentando vender el mismo lote dos veces. |
| Tolerancia | Reintentar automáticamente si la red falla en ese momento. | Para asegurar que el código sí quede quemado. | Viendo los registros del sistema. |

**5. Patrones de Diseño Aplicados**

**Patrón Observer (Comportamiento):** Para que nuestro backend esté escuchando a la blockchain y se actualice solo cuando la venta se confirme.

**6. Limitaciones del Alcance**

Esto no genera facturas legales, eso se hace en la caja registradora del supermercado.

**7. Consideraciones y Notas**

Quemar el código es una transacción de escritura, así que esta sí consume un poco de Gas en la red.

**8. Diseño UI**




Figura 5: Interfaz del punto de venta para confirmar e invalidar el código final.

# Historias de Usuario: Javier Alexis Dejesus Narvaez

## HU-06: Registrar datos de la finca

**Prioridad:** Media

**2. Nomenclatura**

**Como:** Administrador del sistema.
**Quiero:** Registrar dónde queda la finca productora y sus certificados.
**Para:** Saber exactamente de qué tierras viene el aguacate antes de pasarlo a blockchain.

**3. Validaciones (Criterios de Aceptación)**

1. Pedir obligatoriamente el nombre, ubicación GPS y registro del ICA.
2. Si suben un PDF del certificado, que pese máximo 2MB.
3. Que el número de registro de la finca no esté duplicado en la base de datos.
4. Al guardar, la finca queda activa para empezar a registrarle lotes.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Usabilidad | Botón de .ºbtener mi ubicaciónüsando el GPS. | Para que no tengan que escribir coordenadas a mano. | Probando el botón en el celular. |
| Seguridad | Limpiar los campos de texto antes de guardarlos. | Para que no nos hackeen la base de datos con inyección SQL. | Haciendo pruebas de seguridad mandando textos raros. |

**5. Patrones de Diseño Aplicados**

**Patrón Builder (Creacional):** Nos sirve para armar el perfil de la finca poco a poco, porque tiene muchos datos opcionales.
**Patrón Repository:** Para manejar todo el código de guardar y buscar en la base de datos normal.

**6. Limitaciones del Alcance**

No vamos a verificar los PDFs automáticamente en la página del ICA.

**7. Consideraciones y Notas**

Estos datos van en la base de datos SQL normal por temas de privacidad de los dueños, a la blockchain solo va un código ID.

**8. Diseño UI**




Figura 6: Formulario para el registro de datos y certificados de origen de la finca.

### HU-07: Crear e imprimir código QR

**Prioridad:** Alta
**2. Nomenclatura**
**Como:** Empacador.
**Quiero:** Generar un código QR físico.
**Para:** Imprimirlo y pegarlo en la caja para que lo escaneen en el viaje.
**3. Validaciones (Criterios de Aceptación)**

1. Solo se puede generar el QR si el lote de aguacates ya se registró.
2. El QR debe tener la URL de nuestra app y el código del lote.
3. Dejar descargar el código en imagen de buena calidad.

4. No dejar imprimir el mismo QR dos veces por error, a menos que un jefe lo autorice.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Eficiencia | Que el QR se genere en menos de un segundo. | Para no retrasar el trabajo en la bodega. | Probando la descarga rápida. |
| Portabilidad | Usar un nivel de corrección alto en el QR. | Para que se deje leer así la etiqueta se raye un poco. | Rayando un QR impreso y escaneándolo. |

**5. Patrones de Diseño Aplicados**

**Patrón Singleton:** Para asegurarnos de que el generador de códigos internos (IDs) no se repita nunca.

**6. Limitaciones del Alcance**

El sistema solo te da la imagen, pero no se conecta por red a impresoras industriales Zebra.

**7. Consideraciones y Notas**

Se usará una librería sencilla de Node.js para armar la imagen del QR.

**8. Diseño UI**




Figura 7: Interfaz del bodeguero para la generación y asignación del código QR.

### HU-08: Asignar el viaje a un conductor

**Prioridad:** Alta
**2. Nomenclatura**
**Como:** Jefe de bodega.
**Quiero:** Asignarle la caja de aguacates a un transportista.
**Para:** Empezar el despacho y pasarle la responsabilidad al conductor.
**3. Validaciones (Criterios de Aceptación)**

1. Mostrar una lista con los transportistas disponibles.
2. Al despachar, mandar la orden al contrato inteligente.

3. Cambiar el estado del lote a .En Tránsito".

4. Si la blockchain falla, cancelar todo y no dejar el aguacate "volando.en el sistema.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Integridad | Asegurar que la base local y la blockchain guarden exactamente lo mismo. | Para no tener descuadres en el inventario. | Simulando que se cae la luz justo al guardar. |
| Seguridad | Pedir confirmación (como un pin) antes de despachar cosas caras. | Para evitar despachos por error humano. | En la ventanita emergente de confirmación. |

**5. Patrones de Diseño Aplicados**
**Patrón Command:** Para poner las órdenes de envío en una fila por si la red está muy ocupada.
**Patrón Unit of Work:** Para manejar que se guarde bien tanto en la base de datos como en Web3.
**6. Limitaciones del Alcance**
No incluye mapas interactivos para ver por dónde va el camión.
**7. Consideraciones y Notas**
El sistema principal será el que pague el costo (Gas) de esta transacción, no el celular del empleado.
**8. Diseño UI**

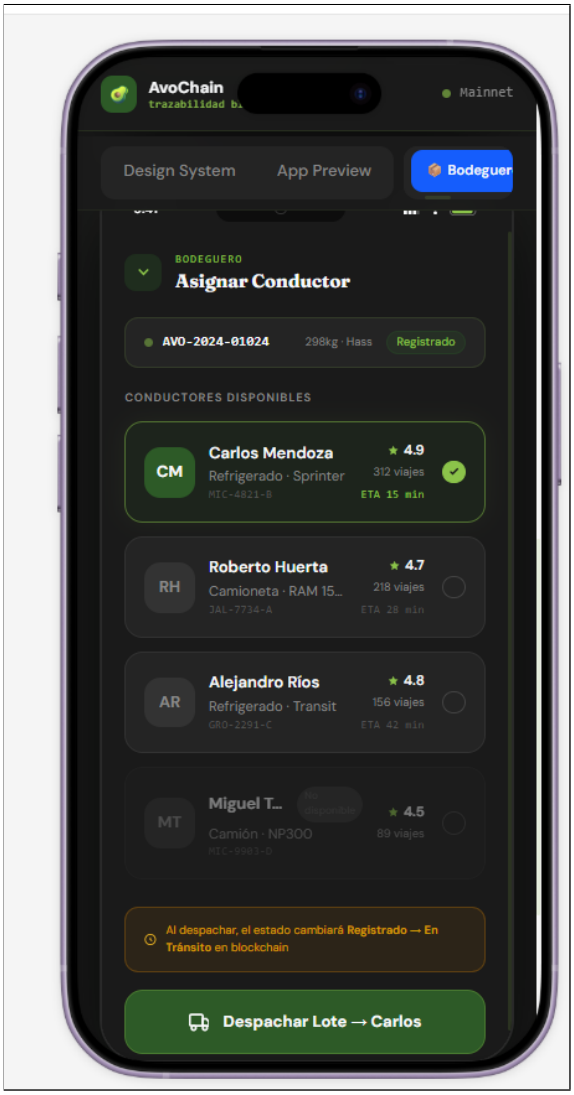



Figura 8: Pantalla de asignación de carga hacia los transportistas disponibles.

### HU-09: Reportar un problema en el viaje

**Prioridad:** Media

**2. Nomenclatura**

**Como:** Transportista.
**Quiero:** Reportar si pasó algo raro en la vía (retrasos, empaque dañado).
**Para:** Explicar por qué el aguacate podría llegar tarde o en mal estado.

**3. Validaciones (Criterios de Aceptación)**

1. Solo se pueden reportar problemas de las cajas que él lleva en su camión.
2. Escoger el problema de una lista y poder escribir un comentario corto.
3. Si escoge "Daño", es obligatorio subir una foto de evidencia.

4. El reporte no se puede borrar ni editar después de guardado.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Eficiencia | Bajarle la calidad a las fotos localmente antes de subirlas. | Para no acabarle los datos del celular al conductor. | Revisando cuánto pesa la foto que llega al servidor. |
| Mantenibilidad | Cargar los tipos de problemas de la base de datos. | Por si mañana queremos agregar un problema nuevo sin actualizar la App. | Revisando de dónde salen las opciones del menú desplegable. |

**5. Patrones de Diseño Aplicados**

**Patrón Decorator (Estructural):** Para agregarle el problema (novedad) al lote original sin dañar el registro base.

**6. Limitaciones del Alcance**

Esto no cancela la venta automáticamente ni avisa a las aseguradoras.

**7. Consideraciones y Notas**

Como son notas cortas, se pueden mandar como .Eventos.en la red blockchain, que salen más baratos.

**8. Diseño UI**




Figura 9: Formulario móvil del conductor para el reporte de incidentes en ruta.

### HU-10: Ver estadísticas de los aguacates

**Prioridad:** Baja
**2. Nomenclatura**
**Como:** Administrador.
**Quiero:** Ver gráficas de cuántos aguacates tenemos registrados, viajando y vendidos.
**Para:** Saber si el negocio va bien y tomar decisiones.
**3. Validaciones (Criterios de Aceptación)**

1. Mostrar números totales divididos por estado.
2. Poner un filtro para buscar estadísticas por fechas.
3. Mostrar una tablita con los últimos 5 problemas reportados por los camiones.

4. Si entra alguien que no es administrador, sacarlo de ahí.

**4. Requerimientos No Funcionales (ISO/IEC 25010)**

| **Categoría ISO 25010** | **Descripción del RNF** | **¿Por qué es necesario?** | **¿Dónde/Cómo se revisará?** |
|---|---|---|---|
| Eficiencia | Que las gráficas carguen leyendo la base de datos local y no la blockchain. | Porque consultar la blockchain para gráficas es lentísimo. | Usando Postman para ver cuánto demora la petición. |
| Tolerancia | Si la librería de gráficas se cae, mostrar al menos los números en texto. | Para que la página no quede en blanco por un error tonto. | Bloqueando el código de gráficas a propósito. |

**5. Patrones de Diseño Aplicados**

**Patrón CQRS:** Lo usamos para separar las gráficas (que solo son lecturas rápidas) de las cosas pesadas que se guardan en la blockchain.

**6. Limitaciones del Alcance**

No incluye un botón para exportar el reporte a Excel o PDF.

**7. Consideraciones y Notas**

Vamos a usar un script o tarea programada para copiar los datos de la blockchain a nuestra base normal, así las gráficas vuelan.

**8. Diseño UI**

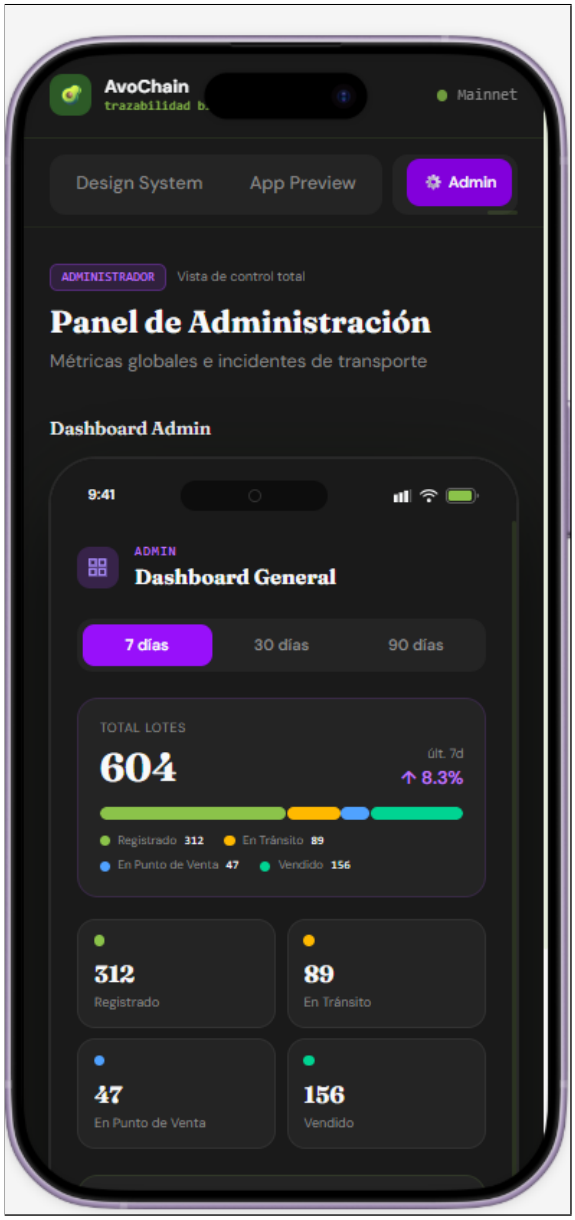



Figura 10: Dashboard de administración con métricas globales del sistema.